Conway's Game of Life



Implementation: CPU (2D arrays) vs. GPU (CUDA)

University of Sri Jayewardenepura
Faculty of Computing

B.Comp (Hons.) in Computer Science
CCS3032 – High-Performance Computing

Requested by: Dr. Budditha Hettige

Group no – 39

Members		
Y.R. Nisal Sanjaya	Index - FC221050	Reg – FC116340
M.D.S.H. Edirisinghe	Index - FC221012	Reg - FC115262

Table of Contents

1. Introduction 3
2. Problem Definition 3
3. Objectives 3
4. Experimental Setup 4
4.1 Platform 4
4.2 Grid and simulation parameters 4
4.3 CPU implementation 5
4.4 GPU implementation 5
4.5 Benchmarking methodology 5
4.6 Correctness verification 5
5. CPU Implementation 7
6. CUDA Implementation 10
7. Comparison of Results 15
7.1 Execution times 15
7.2 Speedup 16
7.3 Analysis 16
7.4 Technical comparison 18
8. Conclusion 18

1. Introduction

Conway's Game of Life is a cellular automaton. Every cell of a grid is either alive or dead, and the grid moves forward in discrete generations. The state of a cell in the next generation depends only on its own current state and the state of the eight cells around it. Simulating a large grid for many generations needs a lot of computation, because every cell must be evaluated in every generation. At the same time, the cells of one generation do not depend on each other, so the problem is a natural candidate for parallel hardware.

In this assignment we implemented Game of Life in two ways:

- a sequential CPU version written in C, using two-dimensional arrays, and
- a parallel GPU version written in CUDA, with one thread for each cell.

Both versions start from the same grid and follow the same rules. We timed 100 generations on a 2048 × 2048 grid and compared the two methods on performance and on technical differences such as the execution model, memory use and data movement.

2. Problem Definition

Every cell of an n × n grid follows the standard rules in each generation:

- A live cell with fewer than 2 live neighbors dies (underpopulation).
- A live cell with 2 or 3 live neighbors survives.
- A live cell with more than 3 live neighbors dies (overpopulation).
- A dead cell with exactly 3 live neighbors becomes alive (reproduction).

The four rules can be written as one expression: next = (n == 3) || (alive && n == 2) where n is the number of live neighbors.

One generation traverses all n^2 cells, counting up to 8 neighbors for each, thus costing $O(n^2)$ work. Cost for g generations is $O(n^2 \times g)$. This is done cell by cell by a single CPU core. The task here is to assign each cell to its own GPU thread, then measure and explain the difference in performance of the two methods.

3. Objectives

- Implement the Game of Life on a 2048 × 2048 grid in C, using 2D arrays, running sequentially on the CPU.
- Implement the same simulation in CUDA, with one GPU thread per cell.
- Give both programs identical starting conditions: the same initial grid generated by the same function, the same density and the same rules.
- Time 100 iterations of each program over repeated runs, after a warm-up run, and report the mean and the range.
- Check that both programs produce the same simulation state.

- Compare the two methods on performance and on technical aspects.
- Identify and explain the techniques that matter for performance on the GPU, and test whether shared-memory tiling helps.
- Produce an animation that shows the CPU and GPU simulations side by side.

4. Experimental Setup

4.1 Platform

All results in this report come from one Kaggle notebook session with a GPU turned on, so the CPU baseline and the GPU programs were measured on the same machine. The notebook prints the hardware and software at the start, and Tables 1 and 2 list them.

Item	Value
CPU model	Intel(R) Xeon(R) CPU @ 2.00GHz
Logical CPUs	4
Threads used by the program	1 (the CPU program is single-threaded)
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compiler flags	-O2 -falign-loops=64

Table 1: Kaggle CPU specifications

Item	Value
GPU	Tesla T4
Compute capability	7.5
Streaming multiprocessors	40
Memory	15,360 MiB
Driver version	580.178.04
CUDA toolkit	release 12.8 (V12.8.93)
Compiler and flags	nvcc -O2 -arch=sm_75

Table 2: Kaggle GPU specifications

4.2 Grid and simulation parameters

- **Grid size:** 2048 × 2048 (4,194,304 cells), one byte per cell
- **Generations per run:** 100
- **Boundary:** wrap-around (toroidal), so the top edge neighbors the bottom edge and the left edge neighbors the right edge
- **Initial state:** pseudo-random with about 30% live cells (1,259,237 of the 4,194,304 cells are alive at generation 0, and 19,696 of the 65,536 cells in the top-left 256 × 256 corner). Each cell value comes from an integer hash of its index and a fixed constant (12345). The same function is used in both programs, so both start from an identical grid

- **Rules:** standard Conway B3/S23, used the same way in both programs
  We chose 2048 × 2048 for two reasons. First, the grid provides the GPU with roughly four million independent threads, which is enough to keep a Tesla T4 busy. Second, it is small enough that the experiment takes about two seconds on the CPU for 100 generations, so it is practical to repeat the experiment.

4.3 CPU implementation

The CPU program keeps the grid in a true two-dimensional array and updates it with nested loops, one cell at a time. It is compiled with the command gcc -O2 -falign-loops=64 -o life_cpu life_cpu.c. The update function is marked noinline and its array parameters are marked restrict (Section 5). The full code is in Section 5.

4.4 GPU implementation

The CUDA program launches one thread per cell in blocks of 16 × 16 threads and one kernel per generation. The grid remains on the GPU for all 100 generations and is swapped between two device buffers via the pointers. It is compiled with **nvcc -O2 -arch=sm_75**. Two kernels were written and compared, a naive kernel that reads neighbors from global memory and a tiled kernel that stages a block of cells in shared memory. The complete code is in Section 6.

4.5 Benchmarking methodology

The CPU program is timed with clock_gettime (CLOCK_MONOTONIC) around the loop of 100 generations. The GPU program timing is done using CUDA events. Within the GPU program, one kernel launch is performed before timing commences and is not timed to avoid counting driver and context startup costs. Two times were recorded for each GPU kernel:

- **Kernel time:** the 100 kernel launches only.
- **Total time:** kernel time plus copying the grid to the GPU and the result back to the CPU.

The benchmark is run as Python cells in the notebook, and nothing else is run during the measurement. Each program is run once before, as a warm-up, and the result is discarded. Then they run five times each, in one process each, first the CPU program and then the GPU program. The mean and the range (from minimum to maximum) of the five runs are reported. The CPU baseline is thus run in the same notebook session as the GPU runs.

The save and frames modes of the programs are run after the benchmark, in separate runs that are not timed, so that writing images does not impact any time reported above.

4.6 Correctness verification

After 100 generations the CPU program ended with 401,781 live cells. Both programs saved their final grids, which were then compared cell by cell. The GPU program also ended with 401,781 live cells, which means that there is a 0 difference between them, and their SHA-256 hashes are the same (c360510d69799c7d…). The two CUDA kernels also resulted in identical

grids, since the GPU program compared them byte by byte and reported MATCH. In Figure 1 one can see the top left 256 × 256 corner of the grids after 100 generations.

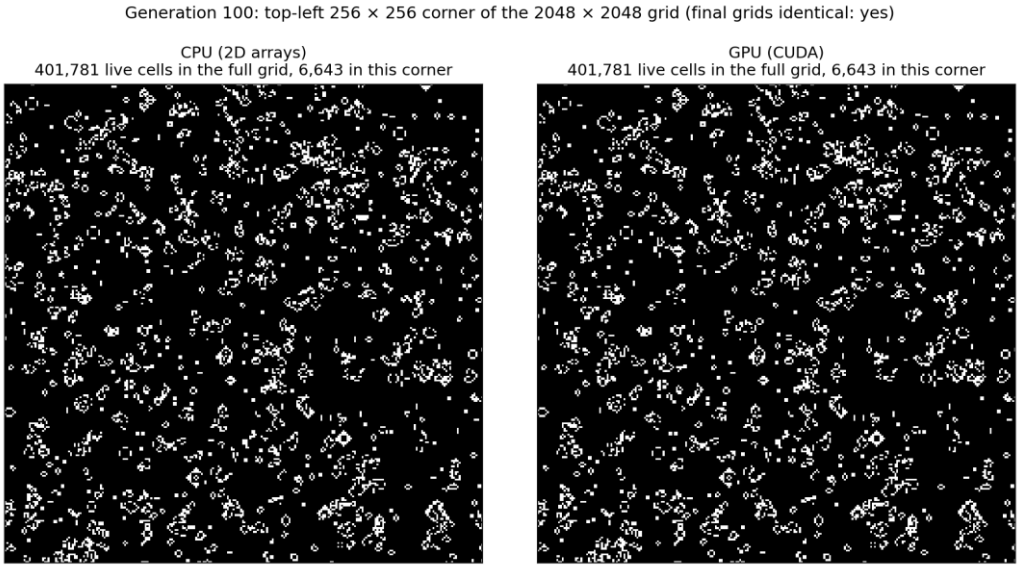


Figure 1: Generation 100 from the CPU program and from the CUDA program

The check was repeated at intermediate generations and not just on the final state, but only for the top-left corner of the grid. Both programs have a frames mode (the CUDA program uses the naive kernel in this mode) whereby the top-left 256 × 256 corner of the grid is saved every second generation, from generation 0 to 100. This provides 51 frames for each program, 51 out of 51 pairs of which were identical byte for byte and Table 3 gives the number of alive cells in this corner every 10 generations (same for both programs). The frames were combined into animated GIFs (life_cpu.gif, life_cuda.gif and life_cpu_vs_cuda.gif – the latter having CPU on the left and CUDA program on the right). Each such GIF has 51 frames with 120 ms per frame. Each 256 × 256 frame was enlarged to 512 × 512 pixels, so that life_cpu.gif and life_cuda.gif are 512 × 512 pixels in size, whereas the side-by-side version is 1032 × 512 pixels (two panels with an 8-pixel gap). Figure 2 contains four of the frames.

Generation	CPU live cells	GPU live cells
0	19,696	19,696
10	14,076	14,076
20	11,172	11,172
30	9,742	9,742
40	8,626	8,626
50	8,192	8,192
60	7,795	7,795
70	7,568	7,568
80	7,256	7,256
90	6,785	6,785
100	6,643	6,643

Table 3: Live cells in the top-left 256 × 256 corner, every 10 generations

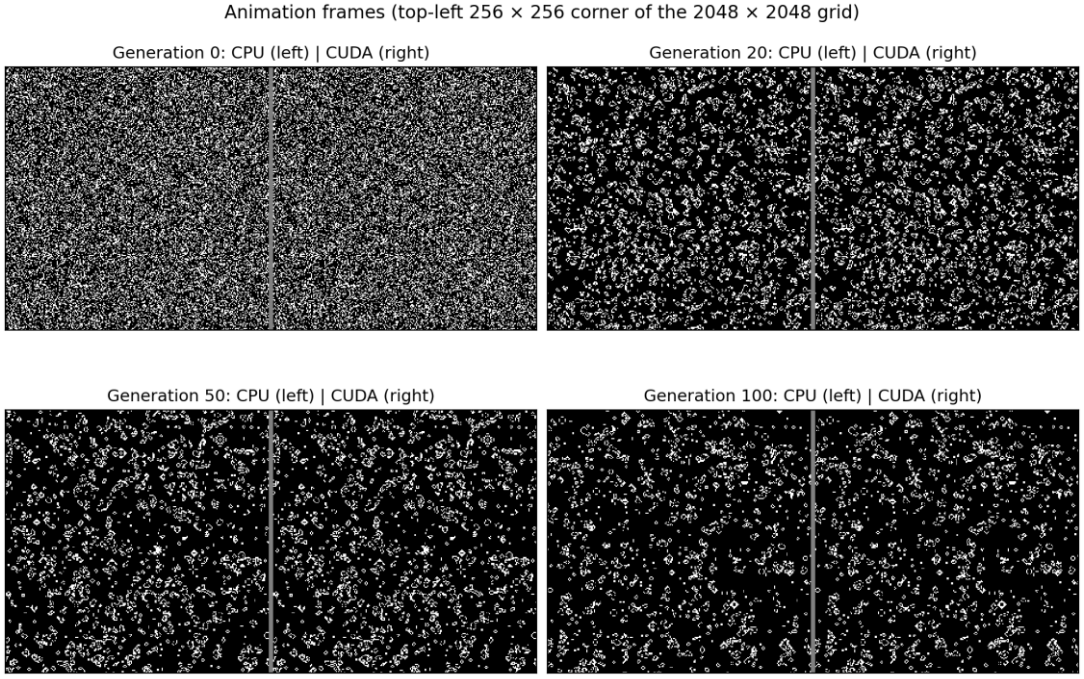


Figure 2: Frames of the animation at generations 0, 20, 50 and 100 (CPU left, CUDA right)

5. CPU Implementation

The CPU version is a straightforward serial implementation which does not utilize any inherent parallelism. The algorithm simply consists of 2 nested loops which iterate through the cells in a row-major order. The latter was chosen because it will allow the inner loop to access consecutive bytes in memory, thus utilizing the CPU cache effectively.

The wrap-around neighbors are found using some index arithmetic. The indices of the rows above and below are computed once per row and the indices of the columns to the left and to

the right once per cell, thus avoiding the need to compute them separately for all 8 neighbors. The new state is written to a separate array and the pointers to these arrays are then swapped at the end of the generation, which takes constant time. This ensures that a cell's new state is not utilized during the same generation.

The original grid is generated from the same hash function the CUDA program employs. Since the steps performed by just one sequential thread are all executed serially, no parallelism can be exploited, so the optimizations performed on the CPU side are limited to reducing the total CPU time spent and ensuring that it's roughly the same across runs. The step function is marked noinline so that its code is placed in a separate function and its performance characteristics are not affected by the rest of the code in the file. Its two array parameters are marked restrict which tells the compiler that the current grid and the next grid will not alias each other. The build process uses -O2 in conjunction with -falign-loops=64 which aligns the beginning of loops to 64-byte boundaries. These optimizations are performed to not artificially inflate the CPU baseline.

There are three modes the program can be run in. The default one, when not invoked with any arguments, just runs and times the simulation. When invoked with the argument save, it also saves the final grid as an image after the timing measurements have finished. When invoked with the argument frames, it runs the step function in the same way but without measuring anything (thus, it's not really useful for timing) and saves the top-left 256x256 corner of the grid as an image every second generation, which can then be used to create the animation.

The CPU implementation is as follows,

```
/*
 * Conway's Game of Life - CPU version (2D arrays)
 * High Performance Computing Assignment
 *
 * Grid : 2048 x 2048, toroidal (wrap-around) boundaries
 * Steps: 100 iterations
 * Build: gcc -O2 -falign-loops=64 -o life_cpu life_cpu.c
 * Run  : ./life_cpu          (timing only)
 *        ./life_cpu save     (also writes the final grid to frame_cpu.pgm)
 *        ./life_cpu frames [k]  (animation mode, NOT timed: writes the top-left
 *                            CROP x CROP corner every k generations, default 2,
 *                            to frames_cpu/frame_GGGG.pgm)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/stat.h>

#define N     2048      /* grid is N x N */
#define ITERS 100
#define SEED  12345u
#define CROP  256       /* size of the corner written in frames mode */

/* Deterministic per-cell hash, identical in the CUDA program,
   so both versions start from the same grid and can be cross-checked. */
static unsigned char init_cell(int r, int c)
{
    unsigned int x = (unsigned int)(r * N + c) ^ SEED;
    x ^= x >> 16; x *= 0x7feb352dU;
    x ^= x >> 15; x *= 0x846ca68bU;
    x ^= x >> 16;
    return (x % 100u) < 30u;           /* ~30% of cells start alive */
}

static double now_ms(void)
```

```
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1000.0 + t.tv_nsec / 1e6;
}

/* noinline: step() stays a separate function, so the timing does not depend on the
   other code in the file. restrict: cur and nxt never overlap. */
static __attribute__((noinline))
void step(unsigned char (*restrict cur)[N], unsigned char (*restrict nxt)[N])
{
    for (int r = 0; r < N; r++) {
        int up = (r == 0)     ? N - 1 : r - 1;
        int dn = (r == N - 1) ? 0     : r + 1;
        for (int c = 0; c < N; c++) {
            int lf = (c == 0)     ? N - 1 : c - 1;
            int rt = (c == N - 1) ? 0     : c + 1;

            int n = cur[up][lf] + cur[up][c] + cur[up][rt]
                  + cur[r][lf]               + cur[r][rt]
                  + cur[dn][lf] + cur[dn][c] + cur[dn][rt];

            nxt[r][c] = (n == 3) || (cur[r][c] && n == 2);
        }
    }
}

/* Write the grid as a binary PGM image: live = white (255), dead = black (0). */
static void save_pgm(const char *path, const unsigned char *g)
{
    FILE *f = fopen(path, "wb");
    if (!f) { fprintf(stderr, "cannot open %s\n", path); return; }
    fprintf(f, "P5\n%d %d\n255\n", N, N);
    unsigned char *buf = (unsigned char *)malloc((size_t)N * N);
    for (size_t i = 0; i < (size_t)N * N; i++) buf[i] = g[i] ? 255 : 0;
    fwrite(buf, 1, (size_t)N * N, f);
    free(buf);
    fclose(f);
}

/* Write only the top-left CROP x CROP corner (used for the animation frames). */
static void save_crop(const char *path, const unsigned char *g)
{
    FILE *f = fopen(path, "wb");
    if (!f) { fprintf(stderr, "cannot open %s\n", path); return; }
    fprintf(f, "P5\n%d %d\n255\n", CROP, CROP);
    for (int r = 0; r < CROP; r++)
        for (int c = 0; c < CROP; c++)
            fputc(g[(size_t)r * N + c] ? 255 : 0, f);
    fclose(f);
}

static long count_alive(unsigned char (*g)[N])
{
    long s = 0;
    for (int r = 0; r < N; r++)
        for (int c = 0; c < N; c++) s += g[r][c];
    return s;
}

int main(int argc, char **argv)
{
    unsigned char (*a)[N] = malloc(sizeof(unsigned char[N][N]));
    unsigned char (*b)[N] = malloc(sizeof(unsigned char[N][N]));
    if (!a || !b) { fprintf(stderr, "malloc failed\n"); return 1; }

    for (int r = 0; r < N; r++)
        for (int c = 0; c < N; c++) a[r][c] = init_cell(r, c);

    printf("Game of Life - CPU (2D arrays)\n");
    printf("Grid: %d x %d  Iterations: %d\n", N, N, ITERS);
    printf("Initial alive cells: %ld\n", count_alive(a));

    /* Animation mode: dump frames, no timing, then exit. */
    if (argc > 1 && strcmp(argv[1], "frames") == 0) {
        int every = (argc > 2) ? atoi(argv[2]) : 2;
        if (every < 1) every = 1;
        mkdir("frames_cpu", 0777);
```

```
        for (int g = 0; g <= ITERS; g++) {
            if (g % every == 0) {
                char name[64];
                snprintf(name, sizeof name, "frames_cpu/frame_%04d.pgm", g);
                save_crop(name, &a[0][0]);
            }
            if (g == ITERS) break;
            step(a, b);
            unsigned char (*t)[N] = a; a = b; b = t;
        }
        printf("Wrote frames every %d generations to frames_cpu/\n", every);
        free(a); free(b);
        return 0;
    }

    double t0 = now_ms();
    for (int i = 0; i < ITERS; i++) {
        step(a, b);
        unsigned char (*t)[N] = a; a = b; b = t;   /* swap buffers */
    }
    double t1 = now_ms();

    double total = t1 - t0;
    printf("Final alive cells  : %ld\n", count_alive(a));
    printf("Total time         : %.3f ms\n", total);
    printf("Time per iteration : %.3f ms\n", total / ITERS);
    printf("Throughput         : %.1f Mcells/s\n",
           (double)N * N * ITERS / (total / 1000.0) / 1e6);

    /* Optional: save the final grid. Done after timing, so timings are unaffected. */
    if (argc > 1 && strcmp(argv[1], "save") == 0) {
        save_pgm("frame_cpu.pgm", &a[0][0]);
        printf("Saved final grid to frame_cpu.pgm\n");
    }

    free(a); free(b);
    return 0;
}
```

6. CUDA Implementation

The CUDA version uses one GPU thread per cell. By launching a kernel and letting the hardware schedule the threads instead of looping over the cells, the program exposes a large amount of parallelism. Each thread calculates its own row and column based on the block and thread indices it belongs to, and proceeds to apply the same neighbor count and rule as the CPU implementation. The rule itself is the same as the CPU version, only the execution differs.

The threads are grouped into blocks of 16 × 16 = 256 threads. This number is a multiple of the 32-thread warp size, so no warp is partially active when the kernel is launched. The 2048 × 2048 grid is covered by 128 × 128 = 16,384 blocks, or 4,194,304 threads in total (Figure 3, right). Threads that are adjacent in a row will have adjacent memory addresses in the array since they are next to each other in the grid. With 16 × 16 blocks, a warp of 32 threads covers two rows of 16 cells, so that their global memory reads form two short contiguous segments that get coalesced into only a few memory transactions.

The grid never leaves the GPU for the duration of the simulation. It is copied to the GPU only once before the generations begin, and the two device buffers are swapped by pointer after each kernel launch. The result is copied back only once at the end of the simulation. Copying the grid back after every generation would be much more expensive than the actual computation that takes place. The main CUDA calls (memory allocation, copies and synchronization) are

enclosed in a CUDA_CHECK macro, because an error from the CUDA runtime otherwise goes unnoticed.

Two kernels were written. The naive kernel simply reads each of the eight neighbors from global memory. The tiled kernel (Figure 3, left) first copies an 18 × 18 tile into shared memory, which consists of the 16 × 16 cells making up the block plus a one-cell wide halo of 68 cells surrounding it. The wrap-around of the grid is applied while the tile is loaded, and then __syncthreads() waits for the whole tile to be ready. Every thread can now read its neighbors from the fast on-chip tile. For 256 output cells the tiled kernel loads 324 bytes from global memory, or about 1.27 loads per cell instead of 9.

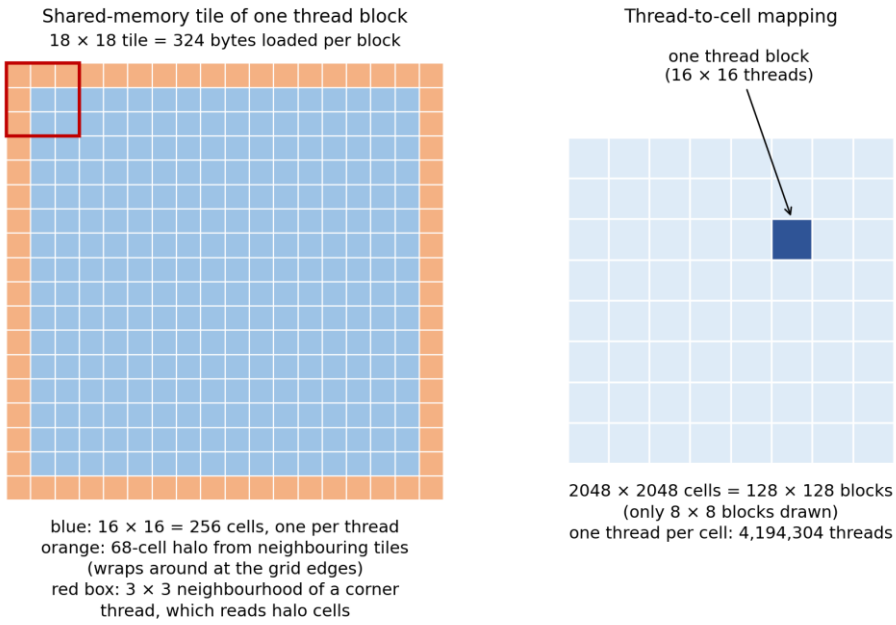


Figure 3: Shared-memory tile with halo (left) and the mapping of threads to cells (right)

The CUDA program has the same three modes as the CPU program. In frames mode it runs the naive kernel without any timing and copies the grid back to the CPU every second generation to write a 256 × 256 corner image. This code is kept separate from the benchmark code, so the timed path is the same as without it.

The CUDA implementation is as follows,

```
/*
 * Conway's Game of Life - CUDA version
 * High Performance Computing Assignment
 *
 * Grid : 2048 x 2048, toroidal (wrap-around) boundaries
 * Steps: 100 iterations
 * Two kernels are timed:
 *   1) naive  - one thread per cell, all neighbours read from global memory
 *   2) tiled  - one thread per cell, block tile + halo staged in shared memory
 *
 * Build: nvcc -O2 -arch=sm_75 -o life_cuda life_cuda.cu
 * Run  : ./life_cuda         (timing only)
 *        ./life_cuda save    (also writes the final grid to frame_cuda.pgm)
 *        ./life_cuda frames [k]  (animation mode, NOT timed: naive kernel, writes
 *                           the top-left CROP x CROP corner every k generations,
 *                           default 2, to frames_cuda/frame_GGGG.pgm)
 */
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <cuda_runtime.h>

#define N     2048
#define ITERS 100
#define SEED  12345u
#define BS    16                        /* thread block = BS x BS */
#define CROP  256                       /* size of the corner written in frames mode */

static_assert(N % BS == 0, "N must be a multiple of the block size");

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t e = (call);                                             \
        if (e != cudaSuccess) {                                             \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(e));                                 \
            exit(1);                                                        \
        }                                                                   \
    } while (0)

/* Same hash as the CPU program so both start from the identical grid. */
static unsigned char init_cell(int r, int c)
{
    unsigned int x = (unsigned int)(r * N + c) ^ SEED;
    x ^= x >> 16; x *= 0x7feb352dU;
    x ^= x >> 15; x *= 0x846ca68bU;
    x ^= x >> 16;
    return (x % 100u) < 30u;
}

/* Write the grid as a binary PGM image: live = white (255), dead = black (0). */
static void save_pgm(const char *path, const unsigned char *g)
{
    FILE *f = fopen(path, "wb");
    if (!f) { fprintf(stderr, "cannot open %s\n", path); return; }
    fprintf(f, "P5\n%d %d\n255\n", N, N);
    unsigned char *buf = (unsigned char *)malloc((size_t)N * N);
    for (size_t i = 0; i < (size_t)N * N; i++) buf[i] = g[i] ? 255 : 0;
    fwrite(buf, 1, (size_t)N * N, f);
    free(buf);
    fclose(f);
}

/* Write only the top-left CROP x CROP corner (used for the animation frames). */
static void save_crop(const char *path, const unsigned char *g)
{
    FILE *f = fopen(path, "wb");
    if (!f) { fprintf(stderr, "cannot open %s\n", path); return; }
    fprintf(f, "P5\n%d %d\n255\n", CROP, CROP);
    for (int r = 0; r < CROP; r++)
        for (int c = 0; c < CROP; c++)
            fputc(g[(size_t)r * N + c] ? 255 : 0, f);
    fclose(f);
}

/* ---------- Kernel 1: naive, global memory only ---------- */
__global__ void life_naive(const unsigned char *in, unsigned char *out)
{
    int c = blockIdx.x * blockDim.x + threadIdx.x;
    int r = blockIdx.y * blockDim.y + threadIdx.y;
    if (r >= N || c >= N) return;

    int up = (r == 0)     ? N - 1 : r - 1;
    int dn = (r == N - 1) ? 0     : r + 1;
    int lf = (c == 0)     ? N - 1 : c - 1;
    int rt = (c == N - 1) ? 0     : c + 1;

    int n = in[up * N + lf] + in[up * N + c] + in[up * N + rt]
          + in[r  * N + lf]                  + in[r  * N + rt]
          + in[dn * N + lf] + in[dn * N + c] + in[dn * N + rt];

    unsigned char alive = in[r * N + c];
    out[r * N + c] = (n == 3) || (alive && n == 2);
}
```

```
/* ---------- Kernel 2: shared-memory tiling with halo ---------- */
__global__ void life_tiled(const unsigned char *in, unsigned char *out)
{
    __shared__ unsigned char tile[BS + 2][BS + 2];

    int tid  = threadIdx.y * BS + threadIdx.x;
    int row0 = blockIdx.y * BS - 1;          /* global row of tile[0][*] */
    int col0 = blockIdx.x * BS - 1;          /* global col of tile[*][0] */

    /* Cooperatively load (BS+2)^2 cells; wrap around at the borders. */
    for (int i = tid; i < (BS + 2) * (BS + 2); i += BS * BS) {
        int tr = i / (BS + 2);
        int tc = i % (BS + 2);
        int gr = (row0 + tr + N) % N;
        int gc = (col0 + tc + N) % N;
        tile[tr][tc] = in[gr * N + gc];
    }
    __syncthreads();

    int ty = threadIdx.y + 1, tx = threadIdx.x + 1;
    int n = tile[ty - 1][tx - 1] + tile[ty - 1][tx] + tile[ty - 1][tx + 1]
          + tile[ty][tx - 1]                        + tile[ty][tx + 1]
          + tile[ty + 1][tx - 1] + tile[ty + 1][tx] + tile[ty + 1][tx + 1];

    int r = blockIdx.y * BS + threadIdx.y;
    int c = blockIdx.x * BS + threadIdx.x;
    out[r * N + c] = (n == 3) || (tile[ty][tx] && n == 2);
}

typedef void (*kernel_t)(const unsigned char *, unsigned char *);

/* Runs ITERS steps with the given kernel. Returns final alive count and
   fills kernel-only time and total time (including H2D + D2H copies). */
static long run(kernel_t k, const unsigned char *h_init, unsigned char *h_out,
                float *kernel_ms, float *total_ms)
{
    size_t bytes = (size_t)N * N;
    unsigned char *d_a, *d_b;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));

    cudaEvent_t s_tot, e_tot, s_k, e_k;
    cudaEventCreate(&s_tot); cudaEventCreate(&e_tot);
    cudaEventCreate(&s_k);   cudaEventCreate(&e_k);

    dim3 block(BS, BS);
    dim3 grid(N / BS, N / BS);

    /* warm-up launch so one-time start-up cost is not measured */
    CUDA_CHECK(cudaMemcpy(d_a, h_init, bytes, cudaMemcpyHostToDevice));
    k<<<grid, block>>>(d_a, d_b);
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEventRecord(s_tot);
    CUDA_CHECK(cudaMemcpy(d_a, h_init, bytes, cudaMemcpyHostToDevice));

    cudaEventRecord(s_k);
    for (int i = 0; i < ITERS; i++) {
        k<<<grid, block>>>(d_a, d_b);
        unsigned char *t = d_a; d_a = d_b; d_b = t;      /* swap buffers */
    }
    cudaEventRecord(e_k);
    CUDA_CHECK(cudaGetLastError());

    CUDA_CHECK(cudaMemcpy(h_out, d_a, bytes, cudaMemcpyDeviceToHost));
    cudaEventRecord(e_tot);
    CUDA_CHECK(cudaEventSynchronize(e_tot));

    cudaEventElapsedTime(kernel_ms, s_k, e_k);
    cudaEventElapsedTime(total_ms, s_tot, e_tot);

    long alive = 0;
    for (size_t i = 0; i < bytes; i++) alive += h_out[i];
```

```
    cudaFree(d_a); cudaFree(d_b);
    cudaEventDestroy(s_tot); cudaEventDestroy(e_tot);
    cudaEventDestroy(s_k);   cudaEventDestroy(e_k);
    return alive;
}

/* Animation mode: naive kernel, copy the grid back every `every` generations and
   write a cropped frame. Not timed, kept separate from run() so the benchmark
   path is unchanged. */
static void run_frames(const unsigned char *h_init, int every)
{
    size_t bytes = (size_t)N * N;
    unsigned char *d_a, *d_b;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    unsigned char *h = (unsigned char *)malloc(bytes);
    CUDA_CHECK(cudaMemcpy(d_a, h_init, bytes, cudaMemcpyHostToDevice));

    dim3 block(BS, BS);
    dim3 grid(N / BS, N / BS);
    mkdir("frames_cuda", 0777);

    for (int g = 0; g <= ITERS; g++) {
        if (g % every == 0) {
            CUDA_CHECK(cudaMemcpy(h, d_a, bytes, cudaMemcpyDeviceToHost));
            char name[64];
            snprintf(name, sizeof name, "frames_cuda/frame_%04d.pgm", g);
            save_crop(name, h);
        }
        if (g == ITERS) break;
        life_naive<<<grid, block>>>(d_a, d_b);
        CUDA_CHECK(cudaGetLastError());
        unsigned char *t = d_a; d_a = d_b; d_b = t;      /* swap buffers */
    }
    printf("Wrote frames every %d generations to frames_cuda/\n", every);
    free(h);
    cudaFree(d_a); cudaFree(d_b);
}

int main(int argc, char **argv)
{
    cudaDeviceProp p;
    CUDA_CHECK(cudaGetDeviceProperties(&p, 0));
    printf("Game of Life - CUDA\n");
    printf("Device: %s (SM %d.%d, %d SMs, %.1f GB)\n", p.name, p.major, p.minor,
           p.multiProcessorCount, p.totalGlobalMem / 1073741824.0);
    printf("Grid: %d x %d  Iterations: %d  Block: %d x %d\n\n", N, N, ITERS, BS, BS);

    size_t bytes = (size_t)N * N;
    unsigned char *h_init  = (unsigned char *)malloc(bytes);
    unsigned char *h_naive = (unsigned char *)malloc(bytes);
    unsigned char *h_tiled = (unsigned char *)malloc(bytes);
    long init_alive = 0;
    for (int r = 0; r < N; r++)
        for (int c = 0; c < N; c++) {
            h_init[(size_t)r * N + c] = init_cell(r, c);
            init_alive += h_init[(size_t)r * N + c];
        }
    printf("Initial alive cells: %ld\n\n", init_alive);

    /* Animation mode: dump frames, no timing, then exit. */
    if (argc > 1 && strcmp(argv[1], "frames") == 0) {
        int every = (argc > 2) ? atoi(argv[2]) : 2;
        if (every < 1) every = 1;
        run_frames(h_init, every);
        free(h_init); free(h_naive); free(h_tiled);
        return 0;
    }

    float k1, t1, k2, t2;
    long a1 = run(life_naive, h_init, h_naive, &k1, &t1);
    long a2 = run(life_tiled, h_init, h_tiled, &k2, &t2);

    double cells = (double)N * N * ITERS;
    printf("%-22s %12s %12s %14s %12s\n", "Kernel", "Kernel (ms)", "Total (ms)",
           "ms/iteration", "Mcells/s");
    printf("%-22s %12.3f %12.3f %14.4f %12.1f\n", "Naive (global mem)", k1, t1,
           k1 / ITERS, cells / (k1 / 1000.0) / 1e6);
```

```
    printf("%-22s %12.3f %12.3f %14.4f %12.1f\n", "Tiled (shared mem)", k2, t2,
           k2 / ITERS, cells / (k2 / 1000.0) / 1e6);

    printf("\nFinal alive cells: naive=%ld tiled=%ld\n", a1, a2);
    printf("Naive vs tiled grids %s\n",
           memcmp(h_naive, h_tiled, bytes) == 0 ? "MATCH" : "DIFFER");
    printf("(CPU reference for this seed: 401781 alive cells)\n");

    /* Optional: save the final grid. Done after timing, so timings are unaffected. */
    if (argc > 1 && strcmp(argv[1], "save") == 0) {
        save_pgm("frame_cuda.pgm", h_naive);
        printf("Saved final grid to frame_cuda.pgm\n");
    }

    free(h_init); free(h_naive); free(h_tiled);
    return 0;
}
```

7. Comparison of Results

7.1 Execution times

Table 4 shows the five CPU runs and Table 5 the five GPU runs. Each time is for 100 generations of the 2048 × 2048 grid, and all of them were measured in the same notebook session after a warm-up run. The mean throughput in Table 4 is calculated from the mean total time, not averaged over the five individual runs.

Run	Total time (ms)	Time per generation (ms)	Throughput (Mcells/s)
1	1826.6	18.27	229.6
2	1876.7	18.77	223.5
3	1903.4	19.03	220.4
4	1940.0	19.40	216.2
5	1925.0	19.25	217.9
Mean	1894.3	18.94	221.4
Range	1826.6 – 1940.0	18.27 – 19.40	216.2 – 229.6

Table 4: CPU (2D array) execution times, 5 runs

Run	Naive kernel (ms)	Naive total (ms)	Tiled kernel (ms)	Tiled total (ms)
1	19.547	24.291	23.933	28.649
2	19.542	24.306	23.942	28.549
3	18.806	23.561	23.415	28.174
4	18.154	22.654	22.949	27.769
5	18.807	23.399	22.945	27.632
Mean	18.971	23.642	23.437	28.155
Range	18.154 – 19.547	22.654 – 24.306	22.945 – 23.942	27.632 – 28.649

Table 5: GPU (CUDA) execution times, 5 runs

7.2 Speedup

The speedup is the mean CPU time divided by the mean GPU time. Table 6 gives it for kernel time alone and for the total time including the copies.

Version	Kernel (ms)	Total (ms)	Per generation, kernel (ms)	Speedup (kernel / total)
CPU, 2D arrays	1894.3	1894.3	18.94	1.0×
CUDA naive	18.971	23.642	0.190	99.9× / 80.1×
CUDA tiled	23.437	28.155	0.234	80.8× / 67.3×

Table 6: Mean times and speedup over the CPU

$$\text{Speedup (naive, kernel)} = 1894.3 \text{ ms} \div 18.971 \text{ ms} \approx 99.9\times$$
$$\text{Speedup (naive, total)} = 1894.3 \text{ ms} \div 23.642 \text{ ms} \approx 80.1\times$$

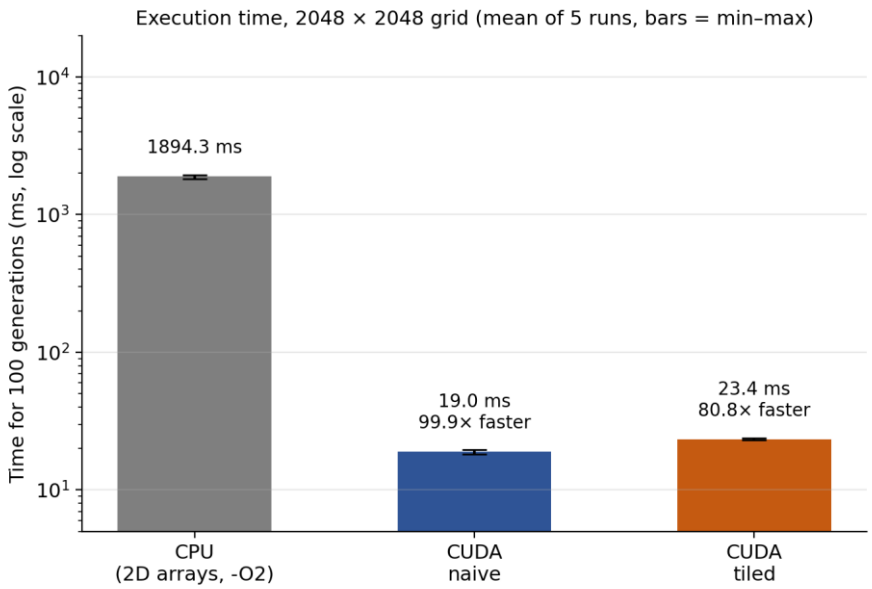


Figure 4: Execution time of the CPU and the two CUDA kernels (log scale; error bars show the minimum and maximum of the five runs)

7.3 Analysis

Run-to-run variation. The five CPU runs lie within about 6% of each other (1826.6 to 1940.0 ms). The first CPU run was the fastest and the later runs were slower, and we did not investigate the reason. The GPU kernel times vary by about 7% for the naive kernel (18.2 to 19.5 ms) and about 4% for the tiled kernel (22.9 to 23.9 ms), and they tended to be slightly lower in the later runs. Here the variation means the difference between the slowest and the fastest run, as a percentage of the mean. A warm-up run of each program was made and discarded before timing, and all five timed runs are included in the means.

Why the GPU is faster. The CPU iterates over one cell at a time on one core, therefore its time grows linearly with the number of cells multiplied by the number of generations. The GPU processes the independent cells of a generation simultaneously via about four million threads,

while the work of each generation is distributed over the 40 streaming multiprocessors of the GPU.

Memory traffic. One generation needs at least one read and one write of the grid, about 8.4 MB, or about 839 MB for 100 generations. The naive kernel took an average of 19.0 ms to complete, which corresponds to an effective bandwidth of about 44 GB/s for this minimum traffic (46 GB/s for the fastest run). Tesla T4 has 320 GB/s of memory bandwidth (according to the manufacturer's specifications), so, counting only this minimum traffic, our kernel used about 14% of the theoretical peak. In other words, the program does not appear to be memory-bandwidth bound. The reason for this is that each thread issues nine one-byte loads, a store, and the index computations, so instruction count per cell probably limits the performance, rather than the number of bytes transferred.

Copy overhead. The copies to and from the GPU took about 4.7 ms for the naive kernel and 4.7 ms for the tiled kernel, which is 20% and 17% of the total time. They are paid once for all 100 generations. This is why the speedup falls from about 100× (kernel only) to about 80× (total) for the naive kernel (Figure 5).

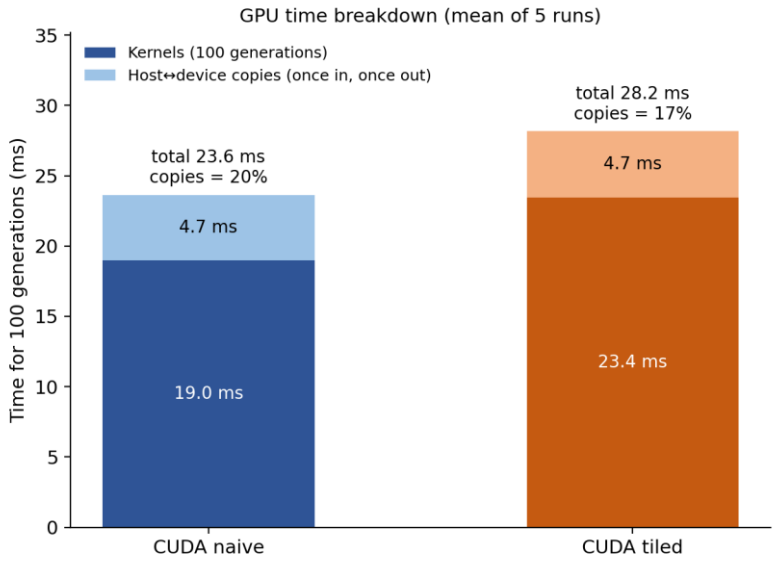


Figure 5: GPU time split into kernel time and copy time (mean of 5 runs)

Naive versus tiled. Tiling decreases the global load from 9 to around 1.27 per cell, but the tiled kernel was slower in all 5 runs, with 23.4 ms vs. 19 ms on average, or by about 23.5%. In individual runs, it was between 22 and 26% slower. The probable reason is that L1 and L2 caches of the T4 GPU already hold many reused elements from the naive kernel; hence, traffic saved by tiling was relatively low. The tiled kernel must perform additional operations: a halo-loading loop, with integer divisions and modulo operations, synchronization barrier, and extra shared memory accesses. Those operations add up to a noticeable overhead, and the misaligned one-byte loads of the halo columns may also contribute. On older GPUs with smaller caches, tiling is usually more beneficial. For this grid and this GPU, it turned out to be overall detrimental, and such cases are possible, hence the optimization was worth testing.

7.4 Technical comparison

Aspect	CPU, 2D arrays (C)	GPU, CUDA
Execution model	Sequential, one thread, nested loops	Data-parallel (SIMT), one thread per cell
Parallelism	None in this version	4,194,304 threads in 16,384 blocks, run in warps of 32
Memory	Registers and L1/L2/L3 caches, managed automatically; then DRAM	Registers, shared memory (programmer-managed), L1/L2, global memory
Data layout	True 2D array, cur[r][c]	Flat 1D array, in[r*N + c]
Memory access	Sequential along rows, helped by the prefetcher	Coalesced across the threads of a warp
Boundary handling	Wrap indices computed per row and column	Wrap computed per thread, or during the tile load
Synchronization	Not needed	Barrier inside a block (tiled kernel); kernel launch between generations
Data movement	None, data stays in host memory	One copy in and one copy out; buffers swapped on the device
Development effort	Low, easy to debug	Higher: kernels, memory management, error checks, launch settings
Portability	Any CPU and C compiler	Needs an NVIDIA GPU and the CUDA toolkit

Table 7: Technical comparison of the two implementations

8. Conclusion

This assignment implemented Conway's Game of Life on a 2048 × 2048 grid in two ways: a sequential CPU version using 2D arrays, and a CUDA one which allocated a thread to each cell. Both versions used the same starting grid and the same ruleset, and both ended with 401,781 live cells after 100 generations; the saved final grids differed in 0 cells, and the 51 saved animation frames of the top-left corner were identical, so the two programs calculated the same simulation.

The CPU took 1894.3 ms on average across 100 generations (about 18.9 milliseconds per generation), while the naive CUDA kernel took 19.0 milliseconds, about 100× faster (about 80× when including the cost of copying data to and from the GPU), and the tiled kernel achieved about 81× and 67×, respectively. The design relies on three techniques: allocating one thread to each cell, which exposes massive parallelism; using blocks of 256 threads (a multiple of the 32-thread warp size) so that neighboring threads make coalesced memory accesses; and keeping the grid on the GPU, so that data is copied only once in each direction rather than every generation. Only the choice between the naive and the tiled memory access was tested experimentally, so this report does not measure how much each technique contributes.

The shared-memory tiling turned out to be ineffective for the Tesla T4 graphics card. The execution time of the tiled kernel proved to be about 23.5% slower than that of the naive one, most likely due to the GPU's caches being already efficient at dealing with the repeated neighbor access. Moreover, the naive kernel only utilized about 14% of the theoretical peak memory bandwidth of the Tesla T4 (counting only the minimum traffic), which suggests that, for the current implementation, optimizing the memory access patterns would provide only limited benefits. Instead, reducing the number of the operations per cell, for instance, by utilizing bitwise operations to store multiple cells in a single word, would likely be more effective, although this was not tested.

It should also be noted that the CPU baseline was single-threaded, and therefore, a multi-threaded version of the CPU code would have provided much better performance, thus reducing the performance gap with the GPU. Since the Kaggle machine has only 4 logical CPUs, the improvement would be at most about 4×, and probably less, so the GPU would still be considerably faster. Moreover, all results were obtained in a single Kaggle session, with one warm-up run followed by five timed runs for each program, so the CPU and the GPU were measured on the same machine. Because Kaggle assigns hardware to each session, results from another session or machine, especially the CPU time, may differ. Nonetheless, the conclusion of the experiment remains valid, namely, that the problem with many independent operations, such as Conway's Game of Life, benefits tremendously from execution on the GPU, provided that the grid is large enough to keep the GPU's cores busy. At the same time, the CPU version of the code remains simple to implement and maintain while providing acceptable performance.